

فایل شماره ۰۴ — نرم افزار

GlassGarden — Software Design & Development

مورد	مقدار
نام پروژه	GlassGarden
نوع پروژه	تراریوم هوشمند رومیزی
نسخه پروژه	1V Prototype
نسخه سند	0.2 (به روزرسانی شده طبق پیشرفت واقعی)
وضعیت	در حال توسعه و تست — سخت افزار اول آسیب دید و در حال تعویض است
آخرین به روزرسانی	۱۴۰۵/۰۵/۱۷ (۲۰۲۶-۰۸-۰۸)

۱. هدف این سند

این سند مرجع اصلی طراحی، توسعه، تست و ادامه کار نرم افزار GlassGarden است. نسخه ۰.۲ این فایل، محتوای نسخه ۰.۱ را با نتایج واقعی به دست آمده از پیاده سازی، تست، و یک حادثه سخت افزاری در همین دوره به روزرسانی می کند.

۲. معرفی کل پروژه

GlassGarden یک تراریوم هوشمند رومیزی است که با ESP32 کنترل می شود و شامل سازه چوبی، محفظه شیشه ای، فضای داخلی گیاهان، سیستم آب (آبشار + پمپ)، مه ساز، فن، نورپردازی، سنسورها، Wi-Fi، Blynk، ESP32 و منطق کنترل خودکار است.

۳. معماری نهایی نرم افزار (پیاده سازی شده)

برخلاف معماری کلی نگر نسخه ۰.۱، معماری نهایی به صورت لایه ای و یک طرفه پیاده سازی شد؛ هیچ کلاسی مستقیماً GPIO را کنترل نمی کند:

- Hardware / Outputs ← تنظیم پین ها و نوشتن دیجیتال + لاگ هر تغییر خروجی
- Device Classes (LightDevice, FanDevice, FoggerDevice, PumpDevice)
- DeviceManager (Global) ← نقطه واحد کنترل: lightOn/Off, fanOn/Off, foggerOn/Off, pumpOn/Off
- StateManager (Global) ← وضعیت لحظه ای: light, fan, fogger, pump, temperature, humidity, wifiConnected, blynkConnected, autoMode
- SensorManager ← خواندن DHT22 هر ۲.۵ ثانیه
- AutomationManager ← تصمیم گیری خودکار (دما/رطوبت/زمان بندی)
- WiFi ← NetworkManager + همگام سازی ساعت NTP
- BlynkManager / BlynkHandlers ← ارتباط دوطرفه با اپ
- SystemManager ← اجرای چرخه begin/update کل سیستم
- main.cpp ← نقطه ورود

۴. محیط توسعه

مورد	مقدار
IDE	VS Code + PlatformIO
Framework	Arduino
برد	esp32dev (ESP32 Dev Module / WROOM-32D)
ارتباط USB	CP2102 — در عمل با دو مشکل درایور (Code 10 و Code 43) و یک کابل معیوب مواجه شد؛ هر دو رفع شدند
سرعت Serial	115200

۵. تنظیمات PlatformIO (نهایی)

board = esp32dev | framework = arduino | monitor_speed = 115200

کتابخانه‌ها: Adafruit Unified Sensor، Adafruit DHT Sensor Library، Blynk/Blynkkk (طبق platformio.ini فعلی) — بدون نیاز به تغییر تا این مرحله).

۶. ساختار نهایی فایل‌های پروژه

ساختار واقعی، دقیق‌تر و ماژولارتر از پیشنهاد اولیه نسخه ۰.۱ اجرا شد:

- src/core/ ← Config.h, Datastreams.h
- *.src/hardware/ ← Hardware.*, Outputs (شامل لاگ‌گیری هر خروجی)
- *.src/devices/ ← LightDevice.*, FanDevice.*, FoggerDevice.*, PumpDevice.*, DeviceManager
- *.src/state/ ← StateManager
- *.src/sensors/ ← SensorManager
- *.src/automation/ ← AutomationManager
- *.src/network/ ← NetworkManager
- src/blynk/ ← BlynkManager.*, BlynkHandlers.cpp
- *.src/system/ ← SystemManager
- src/main.cpp

۷. GPIO Mapping

نوع	تجهیز	نام منطقی	پین ESP32
Output	Light	LIGHT_PIN	23
Output	Fogger	FOGGER_PIN	22
Output	Fan	FAN_PIN	21
Output	Pump	PUMP_PIN	19
Input	DHT22 (DHT_TYPE=22)	DHT_PIN	4

۸. Config.h — وضعیت نهایی

تمام مقادیر سخت‌افزاری/شبکه/اتوماسیون در Config.h متمرکز شدند، دقیقاً طبق اصل نسخه ۰.۱. مقادیر واقعی WiFi و Blynk طبق قانون امنیتی همین سند (بخش ۲۰) در این فایل عمومی درج نمی‌شوند و فقط در نسخه محلی پروژه روی سیستم کاربر نگهداری می‌شوند.

- WIFI_SSID، WIFI_PASSWORD ← تکمیل‌شده (مقدار محرمانه، فقط لوکال)
- BLYNK_TEMPLATE_ID، BLYNK_TEMPLATE_NAME، BLYNK_AUTH_TOKEN ← تکمیل‌شده (مقدار محرمانه، فقط لوکال)
- LIGHT_PIN=23، FOGGER_PIN=22، FAN_PIN=21، PUMP_PIN=19، DHT_PIN=4، DHT_TYPE=22
- OUTPUT_ACTIVE_HIGH=true ← تأیید شد که با ماژول‌های MOSFET انتخاب‌شده سازگار است
- AUTOMATION_ENABLED=true + آستانه‌های دما/رطوبت/زمان‌بندی هر تجهیز (بخش ۱۱)
- NTP_SERVER=pool.ntp.org، آفست منطقه زمانی ۳:۳۰+ (ایران)

۹. Blynk Virtual Pins — نهایی

وضعیت	نوع Widget	Data Type	نام	Pin
ساخته و تست شد	Switch	Integer (0/1)	Light	V0
ساخته شد	Value Display / Gauge	Double	Temperature	V1
ساخته شد	Value Display / Gauge	Double	Humidity	V2
ساخته شد	Switch	Integer (0/1)	Fogger	V3
ساخته شد	Switch	Integer (0/1)	Fan	V4
ساخته شد	Switch	Integer (0/1)	Pump	V5
اضافه‌شده در این نسخه	Switch	Integer (0/1)	Auto Mode	V6

برخلاف نسخه ۰.۱ که V1/V2 (نمایش دما/رطوبت) را «نیازمند تکمیل» اعلام کرده بود، این دو Datastream و Widget مربوط به دما و رطوبت در Blynk.Console ساخته و تأیید شدند.

۱۰. منطق کنترل خروجی و DeviceManager

اصل «کنترل فقط از طریق تابع مشخص، نه digitalWrite پراکنده» به‌طور کامل رعایت شد. تمام دستورات (چه از Blynk، چه از AutomationManager) فقط از طریق متدهای (lightOn/Off، fanOn/Off، foggerOn/Off، pumpOn/Off) DeviceManager به سخت‌افزار می‌رسند.

۱۱. AutomationManager — منطق نهایی (به جای placeholder نسخه ۰.۱)

تجهیز	منطق نهایی (با Hysteresis)
فن	دما $\leq 28^{\circ}\text{C}$ یا رطوبت $\leq 75\%$ یا بازه ساعت ۸ تا ۲۰ ← روشن دما $\geq 26^{\circ}\text{C}$ و رطوبت $\geq 70\%$ و خارج از بازه خاموش ← خاموش
مه‌ساز	رطوبت $\geq 60\%$ یا بازه ساعت ۶ تا ۲۲ ← روشن رطوبت $\leq 70\%$ و خارج از بازه خاموش ← خاموش

تجهیز	منطق نهایی (با Hysteresis)
روشنایی	فقط بازه ساعت ۸ تا ۲۰
پمپ	فقط بازه ساعت ۸ تا ۲۰ (کاربرد: پمپ آبشار، نه آبیاری)

این پیاده‌سازی نیاز بخش «قانون مهم Automation» نسخه ۰.۱ (جلوگیری از روشن/خاموش سریع) را با فاصله‌گذاری ON/OFF (Hysteresis) برآورده می‌کند؛ مدیریت Minimum ON/OFF Time هنوز اضافه نشده (بخش ۲۴، تصمیمات باز).

۱۲. حالت‌های عملکرد سیستم — تصمیم نهایی

به‌جای طرح سه‌حالتی MANUAL / AUTOMATIC / SAFE در نسخه ۰.۱، یک سویچ سراسری Virtual Pin AUTO/MANUAL (V6) پیاده‌سازی شد: در حالت MANUAL، AutomationManager هیچ تصمیمی نمی‌گیرد و کنترل کامل با کاربر است؛ در حالت AUTO، منطق بخش ۱۱ اجرا می‌شود. حالت Safe هنوز پیاده‌سازی نشده (تصمیم باز).

۱۳. اولویت کنترل — تصمیم نهایی

تصمیم باز نسخه ۰.۱ درباره اولویت بین فرمان‌های متناقض حل شد: چون در حالت MANUAL کل AutomationManager غیرفعال می‌شود، اصلاً فرمان متناقضی رخ نمی‌دهد — اولویت به‌صورت «سویچ‌محور» (نه لایه‌ای SAFE → MANUAL → AUTOMATIC) طراحی شد.

۱۴. تایمرها و حلقه اصلی

طبق اصل نسخه ۰.۱، از delay() در مسیر اصلی استفاده نشد. حلقه واقعی SystemManager::update() به این ترتیب اجرا می‌شود:

- network.update()
- devices.update()
- sensors.update() (هر ۲.۵ ثانیه با millis)
- automation.update()
- state.update()
- blynk.update() (تلاش اتصال هر ۵ ثانیه + ارسال دوره‌ای هر ۵ ثانیه)

۱۵. راه‌اندازی اولیه و وضعیت اولیه خروجی‌ها

ترتیب واقعی setup: Hardware::begin → state.begin → devices.begin → sensors.begin → automation.begin → network.begin → blynk.begin. تمام خروجی‌ها هنگام بوت OFF مقداردهی می‌شوند (Outputs::allOff) در Hardware::begin، مطابق اصل «تجهیزات نباید هنگام Boot بدون دلیل روشن شوند».

۱۶. مدیریت خطای سنسور، WiFi و Blynk

- DHT: در صورت isnan بودن مقدار خوانده‌شده، مقدار قبلی state حفظ می‌شود و خطا فقط Log می‌شود ([Sensor] Read «failed»).
- WiFi: قطعی WiFi باعث توقف اتوماسیون یا GPIO نمی‌شود؛ NetworkManager هر ۱۰ ثانیه تلاش مجدد می‌کند.
- Blynk: قطعی Blynk مستقل از خواندن سنسور و کنترل محلی است؛ BlynkManager هر ۵ ثانیه تلاش مجدد می‌کند.

۱۷. Logging

فرمت واقعی پیاپی‌سازی شده (نمونه واقعی گرفته شده از Serial Monitor):

```
Connecting... → [WiFi] Connected → [WiFi] IP : ... → [Time] NTP sync requested → [Blynk] Connecting... → [WiFi]
[Blynk] Connected → [Output] Light (GPIO 23) -> ON
```

۱۸. حادثه سخت‌افزاری — سوختن برد اول ESP32 (مورد جدید، ثبت نشده در نسخه ۰.۱)

در جریان تست سیم‌کشی مدار ۱۲ ولت (روشنایی LED از طریق مازول MOSFET)، برد اول (ESP32-DevKitC + CP2102) بعد از حدود ۱ دقیقه از اتصال داغ شد (~۵۰°C زیر چیپ) و ارتباط قطع شد.

مراحل عیب‌یابی:

- تست با برد کاملاً ایزوله (بدون هیچ سیم اضافه) → باز هم داغ و قطع → رد شد فرضیه «مشکل از سیم‌کشی خارجی»
- تست مولتی‌متر بین 5V/VIN و GND (بدون هیچ برقی) → مقاومت ۰ اهم (اتصال کوتاه کامل)

نتیجه نهایی:

برد دچار آسیب سخت‌افزاری دائمی شد (رگولاتور ولتاژ یا مسیر داخلی سوخته) و باید تعویض شود. این حادثه ربطی به کد یا معماری نرم‌افزار نداشت (طبق بخش ۳۵ نسخه ۰.۱ که این احتمال را مطرح کرده بود، بررسی شد و رد شد).

اقدامات پیشگیرانه برای برد جدید:

- سیم‌کشی فقط با منبع تغذیه خاموش/جدا انجام شود
- هیچ سیمی از ترمینال V+/V-/VIN مدار ۱۲ ولت مستقیم به پین‌های ESP32 وصل نشود
- فقط GND بین ESP32 و منبع ۱۲ ولت مشترک باشد
- قبل از اتصال ESP32، مدار ۱۲ ولت با مولتی‌متر از نظر اتصال کوتاه چک شود
- انتخاب نهایی قطعه سوئیچینگ: ترانزیستور مجزای IRLZ44N (Logic-Level، سازگارتر با ۳.۳ GPIO ولت) به جای مازول‌های آماده IRF520، به همراه مقاومت گیت ۱۰kΩ Pull-down، ۲۲۰Ω، و دیود فلای‌بک 1N4007 برای بارهای موتوردار (فن و پمپ)

۱۹. خطاهای ثبت شده نرم‌افزاری و سخت‌افزاری (به‌روزشده)

ID	خطا	راه حل / وضعیت
SW-003	uint32_t / 'uint8_t' does not name a type	رفع شد — با <code><include <Arduino.h></code> در ابتدای Config.h
SW-004	multiple definition of setup()	رفع شد — قانون جداسازی تست‌ها در Sandbox حفظ شد، پروژه نهایی فقط یک main.cpp دارد
SW-007	error Please specify # BLYNK_TEMPLATE_ID/NAME	رفع شد — علت واقعی: ترتیب اشتباه <code>include</code> منجر به تعریف‌نشدن BLYNK_TEMPLATE_ID قبل از هدر Blynk شد
SW-008	linker error: multiple definition of `Blynk`	رفع شد — در BlynkHandlers.cpp به جای BlynkSimpleEsp32.h، فقط BlynkApiArduino.h سبک‌تر <code>include</code> شد
SW-009	Config.h در redefinition خطای (تکرار محتوای Paste شده)	رفع شد — کل فایل به جای Paste جزئی، کامل جایگزین شد
HW-001	Windows Code 43 — device descriptor request failed	رفع شد — تعویض کابل USB (کابل فقط‌شارژ بود)
HW-002	Windows Code 10 — this device cannot start	رفع شد — نصب CP210x Universal Windows Driver از طریق فایل inf.

ID	خطا	راه حل / وضعیت
HW-003	PermissionError(13) / Error 31 — could not open COM3	رفع شد — تعویض کابل USB
HW-004	داغ شدن (ESP32 ~50°C) و قطع ارتباط بعد از ۱ دقیقه	برد آسیب دیده و در حال تعویض — جزئیات در بخش ۱۸

۲۰. امنیت Configuration

طبق قانون نسخه ۰.۱، مقادیر WIFI_PASSWORD و BLYNK_AUTH_TOKEN در این سند عمومی درج نشدند و فقط در Config.h محلی روی سیستم کاربر نگهداری می‌شوند.

۲۱. تصمیمات تأیید شده (به‌روزشده)

ID	موضوع	تصمیم	وضعیت
S-001	MCU	ESP32	تأیید
S-002	Framework	Arduino	تأیید
S-003	IDE	PlatformIO / VS Code	تأیید
S-004	ارتباط	Wi-Fi	تأیید
S-005	Control	Blynk Cloud	تأیید
S-006	DHT Pin	GPIO4	تأیید
S-007	Light	GPIO23	تأیید
S-008	Fogger	GPIO22	تأیید
S-009	Fan	GPIO21	تأیید
S-010	Pump	GPIO19	تأیید
S-011	اولویت کنترل	سوئیچ AUTO/MANUAL روی V6 (نه سه‌حالت) (SAFE/MANUAL/AUTOMATIC)	تأیید
S-012	منطق Automation	آستانه دوگانه (Hysteresis) برای فن و مه‌ساز	تأیید
S-013	قطعه سوئیچینگ	IRLZ44N گسسته + مقاومت/دیود دستی	تأیید
S-014	سنسور	DHT22 (نه DHT11)	تأیید

۲۲. تصمیمات باز (به‌روزشده)

- Safe Mode هنوز تعریف/پیاده‌سازی نشده
- Minimum ON/OFF Time برای اتوماسیون (فقط Hysteresis ساده پیاده شده، نه محدودیت زمانی)
- مدیریت OTA — برای آینده
- ثبت Log دائمی (روی SD Card یا سرور) — برای آینده
- قابلیت تنظیم آستانه‌ها/زمان‌بندی از داخل اپ (فعلاً فقط در Config.h)

۲۳. وضعیت فعلی توسعه (به‌روزشده)

وضعیت	بخش
آماده	PlatformIO / Arduino Framework
آسیب‌دیده، در حال تعویض	ESP32 Board
ساخته و تکمیل شده	Config.h
تعریف و تست‌شده روی برد اول	GPIO Mapping
کامل (V0-V6)	Blynk Dashboard
ایجاد و کدنویسی شده	Manual Controls
کدنویسی شده، منتظر سنسور فیزیکی	DHT
پیاده‌سازی و کدنویسی کامل، تست فیزیکی باقی‌مانده	Automation
پایه پیاده‌سازی شده (سنسور WiFi/Blynk/ مستقل از هم)	Error Handling
ایجاد نشده	Safe Mode
WiFi+Blynk با موفقیت تست شد؛ تست فیزیکی خروجی‌ها باقی‌مانده	تست نهایی

۲۴. تست‌های واقعی انجام‌شده تا امروز

- Build کامل بدون خطا ✓
- Upload موفق روی برد اول ✓ (قبل از حادثه سخت‌افزاری)
- اتصال WiFi موفق ✓ (لاگ: Connected, IP [WiFi] دریافت شد)
- همگام‌سازی NTP انجام شد ✓
- اتصال Blynk موفق ✓ (لاگ: Connected [Blynk])
- لاگ‌گیری خروجی افزوده و آماده شد؛ تست فیزیکی روشن‌شدن LED به‌خاطر حادثه برد ناتمام ماند ☹️
- تست فیزیکی DHT / Pump / Fogger / Fan هنوز انجام نشده ⌚

۲۵. مرحله بعدی دقیق

- ESP32 (پیشنهاد: نسخه USB Native مثل ESP32-S3/C3-DevKitC برای پرهیز از دردسر درایور CP210x) □□□□□□ □□□ □□□□□□
- ساخت مدار IRLZ44N برای هر ۴ خروجی طبق شماتیک تأییدشده (بخش ۱۸)، با رعایت کامل چک‌لیست ایمنی
- تست تک‌به‌تک هر خروجی با روش لاگ («[Output] ... -> ON/OFF») پیش از اتصال هم‌زمان همه
- تهیه و اتصال سنسور DHT22 فیزیکی
- اجرای تست طولانی‌مدت یکپارچه (WiFi + Blynk + Automation) همه خروجی‌ها هم‌زمان
- پیاده‌سازی Safe Mode □ Minimum ON/OFF Time (تصمیمات باز)

۲۶. ثبت تغییرات سند

نسخه	تاریخ	تغییر
0.1	۲۰۲۶-۰۸-۰۸	ایجاد سند نرم افزار
0.2	۲۰۲۶-۰۸-۰۸	به روز رسانی کامل طبق پیاده سازی واقعی: معماری نهایی، منطق Automation، Virtual Pin V6. حادثه سخت افزاری برد اول، رفع ۹ خطای ثبت شده، تصمیمات جدید تأیید شده، وضعیت و مراحل بعدی

پایان فایل شماره ۰۴ — GlassGarden — Software Design & Development (نسخه سند 0.2)